
Les échecs à l'envers

Comment résoudre algorithmiquement des problèmes
de mat en N coups aux échecs ?

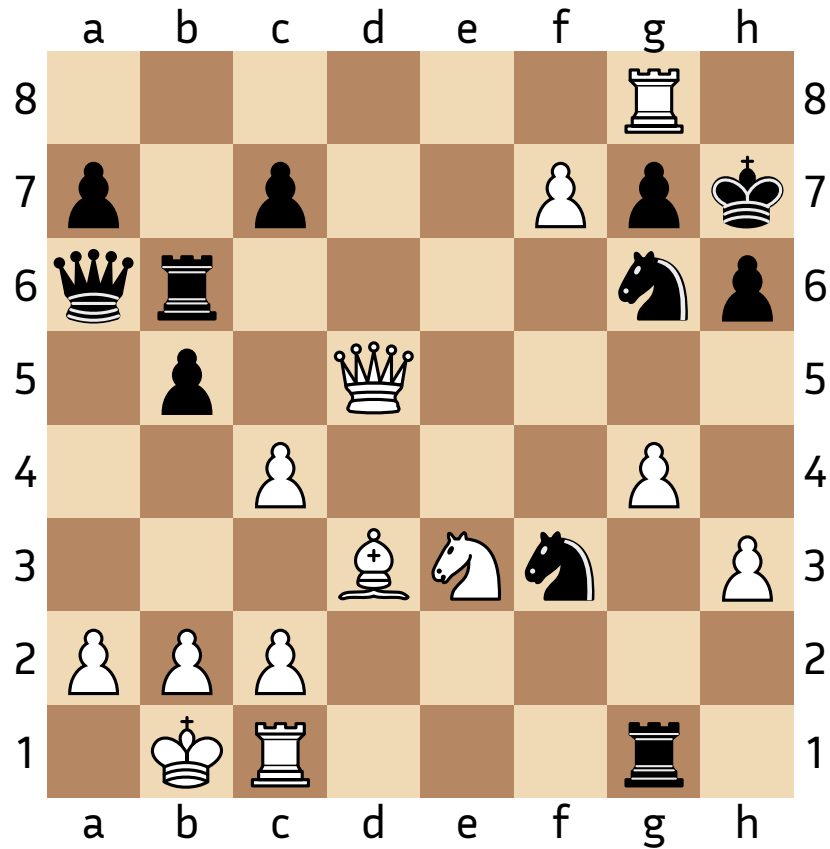
Mathias ANSEAUME 45162



Introduction

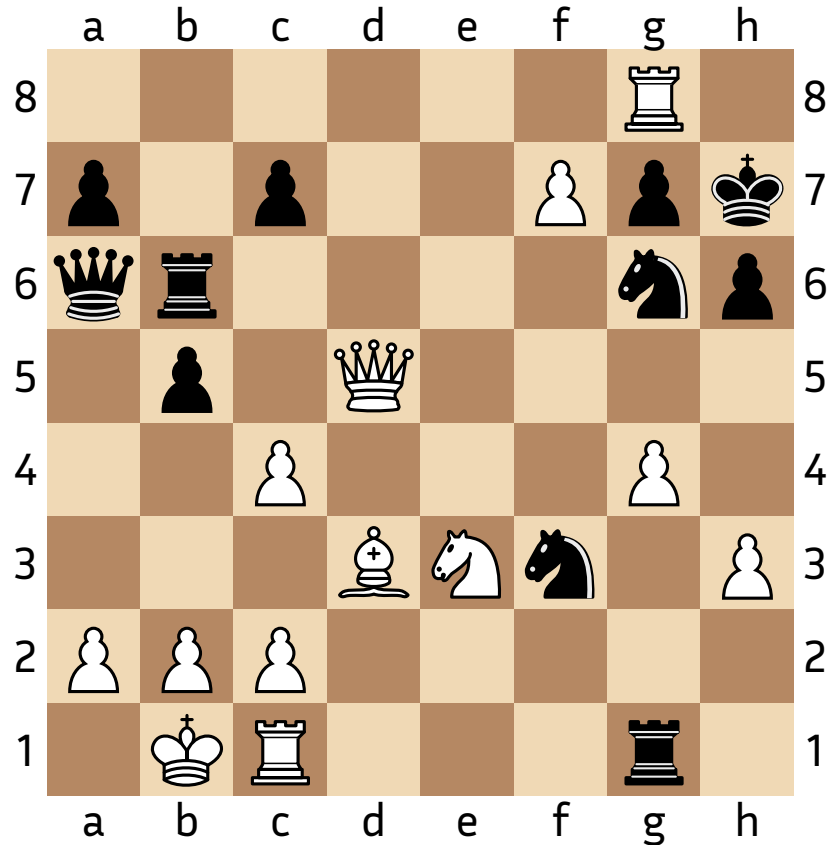
- Génération des coups légaux
- Recherche du mat
- Fins de parties
- Conclusion

Représentation de la position



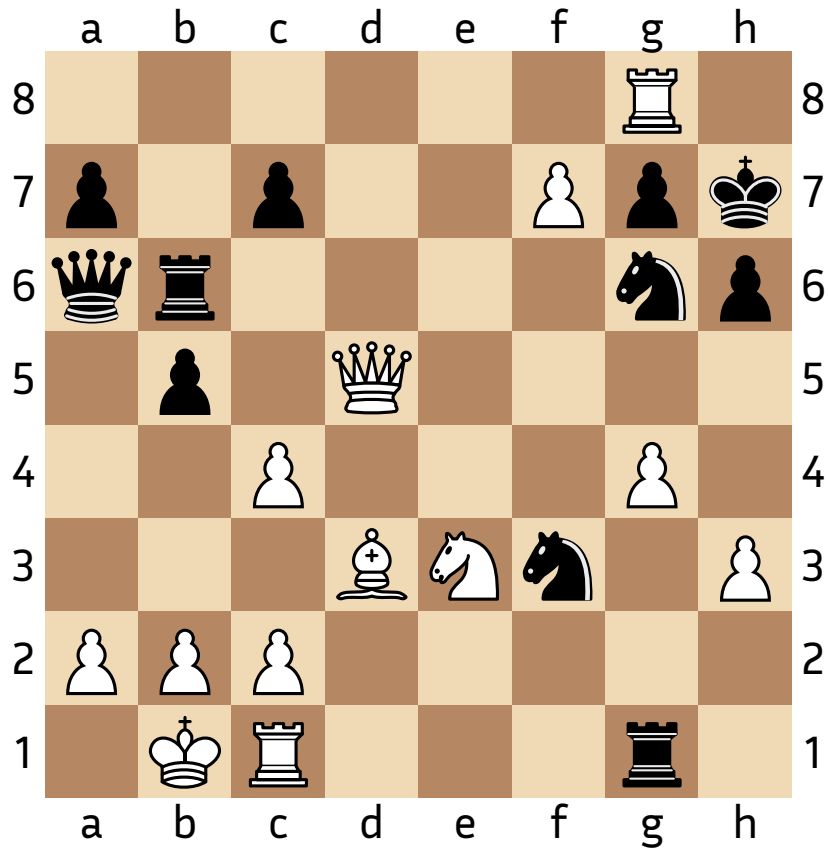
$$\begin{pmatrix}
 . & . & . & . & . & . & t_b & . \\
 p_n & . & p_n & . & . & p_b & p_n & r_n \\
 d_n & t_n & . & . & . & . & c_n & p_n \\
 . & p_n & . & d_b & . & . & . & . \\
 . & . & p_b & . & . & . & p_b & . \\
 . & . & . & f_b & c_b & . & . & p_b \\
 p_b & p_b & p_b & . & . & . & . & . \\
 . & r_b & t_b & . & . & . & t_n & .
 \end{pmatrix}$$

Représentation de la position



$$\begin{pmatrix} \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & t_b & \cdot \\ p_n & \cdot & p_n & \cdot & \cdot & p_b & p_n & r_n \\ d_n & t_n & \cdot & \cdot & \cdot & \cdot & c_n & p_n \\ \cdot & p_n & \cdot & d_b & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & p_b & \cdot & \cdot & \cdot & p_b & \cdot \\ \cdot & \cdot & \cdot & f_b & c_b & \cdot & \cdot & p_b \\ p_b & p_b & p_b & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & r_b & t_b & \cdot & \cdot & \cdot & t_n & \cdot \end{pmatrix} = \sum_p p \cdot M_p$$

Représentation de la position

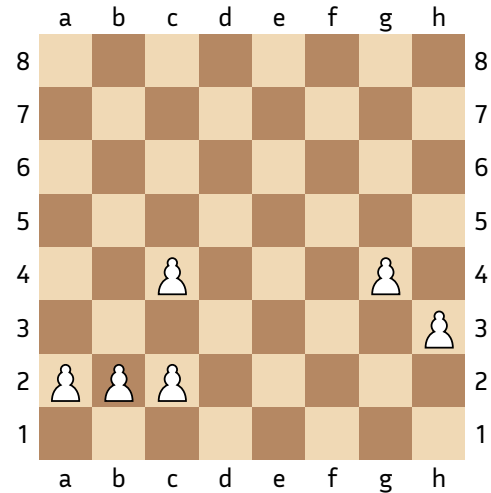


$$M_{p_b} = \begin{pmatrix} \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & 1 & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & 1 & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & 1 & \cdot & \cdot & \cdot & 1 & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & 1 \\ 1 & 1 & 1 & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \end{pmatrix}$$

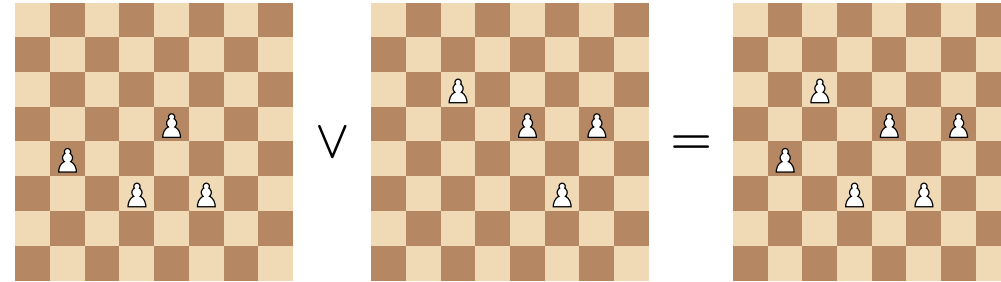
$$M_{c_n} = \begin{pmatrix} \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & 1 & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & 1 & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \end{pmatrix}$$

Structure de données

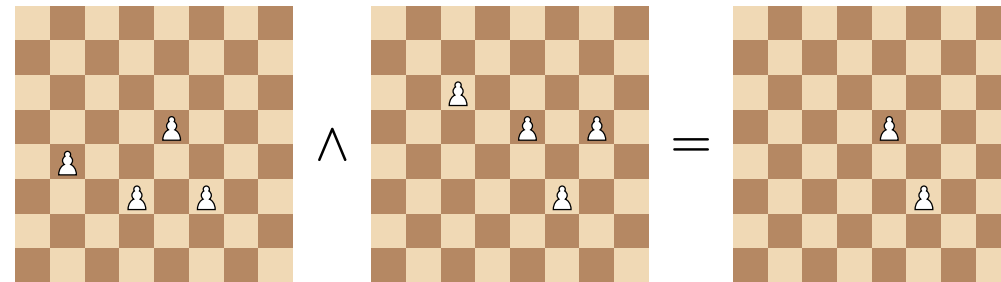
- Module Int64 OCaml



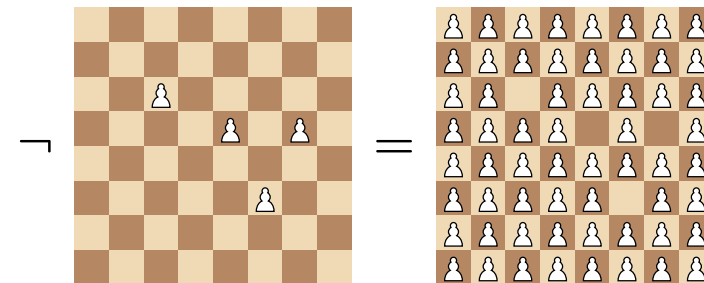
Disjonction



Conjonction



Négation



Generations des coups du cavalier

```

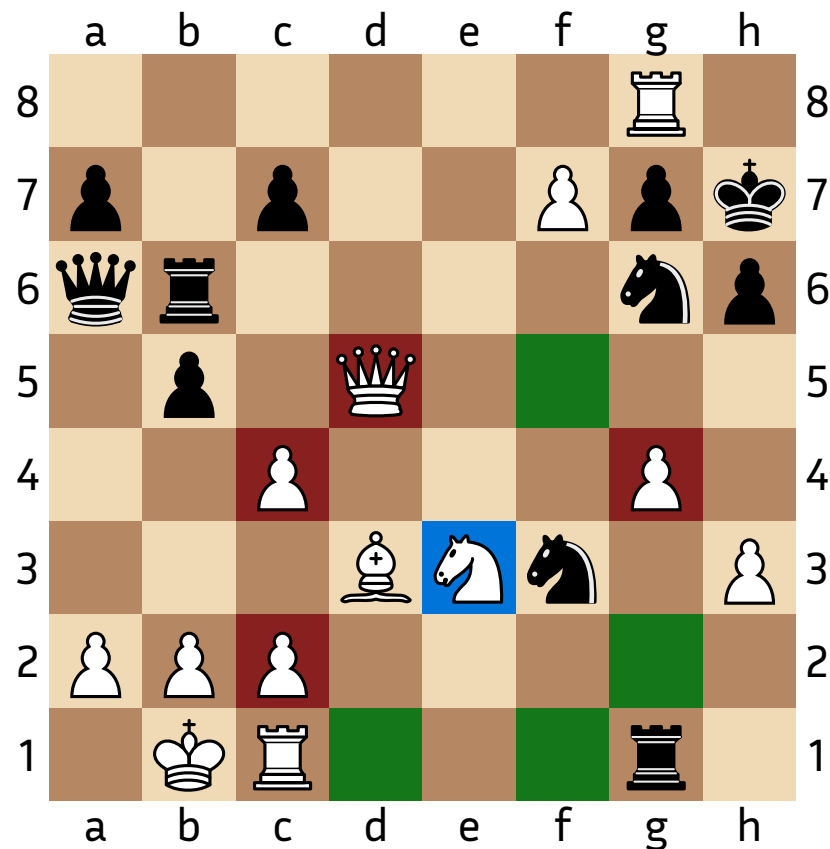
tableau_coups = Nouveau tableau de taille 64
Pour n de 0 jusqu'à 63
  i, j = n / 8, n mod 8
  tableau_coups[i] = bitboard([
    (i+1, j-2), (i+2, j-1), (i+2, j+1), (i+1, j+2),
    (i-1, j+2), (i-2, j+1), (i-2, j-1), (i-1, j-2)
  ])
Fin Pour

```

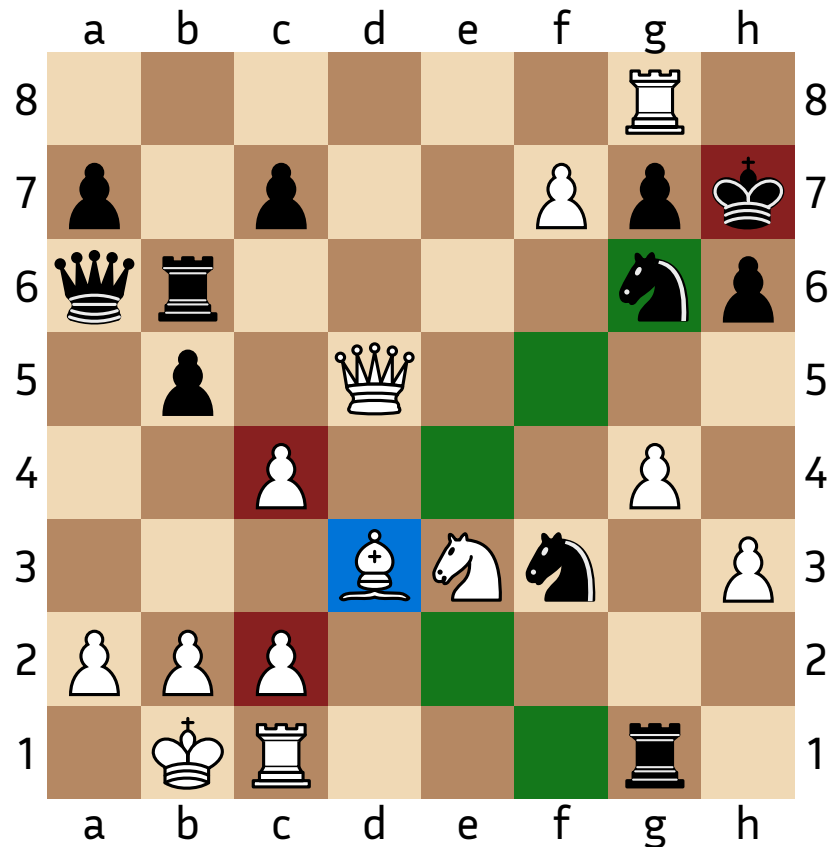
```

Fonction generer_coups echiquier
  coups = []
  Pour chaque cavalier blanc de echiquier
    n = indice cavalier
    coups.ajouter(tableau_coups[n] ^ ~(pieces blanches))
  Fin Pour
Fin Fonction

```



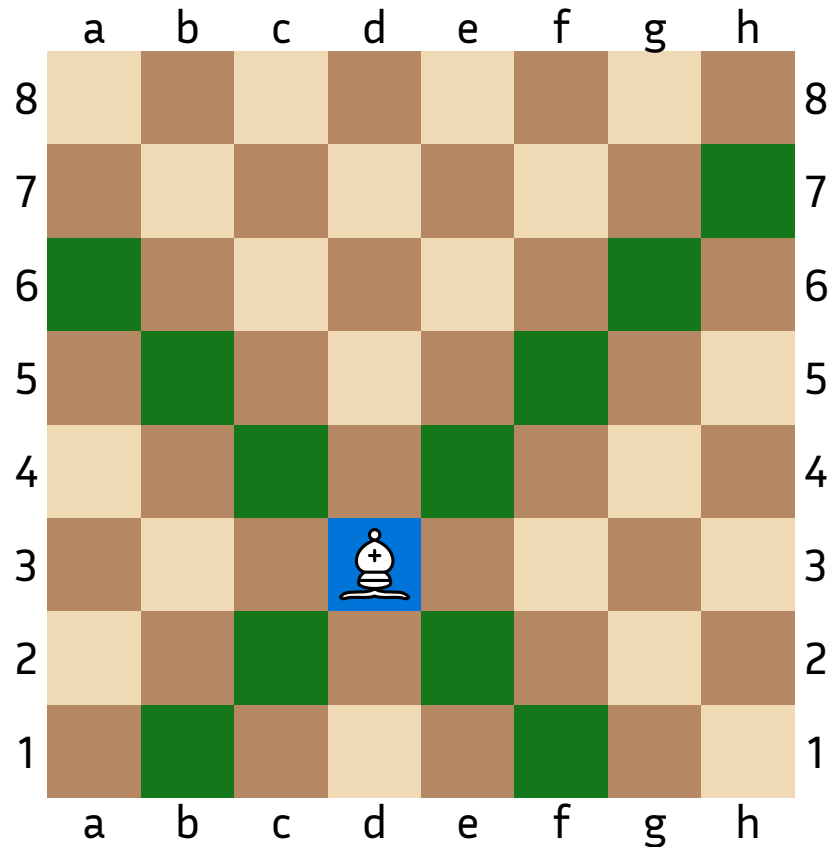
Generation des coups du fou



Problématiques:

- pièce qui glisse $\Rightarrow$ bloqueurs possibles
- bloqueur $\Rightarrow$ cases derrière inaccessibles

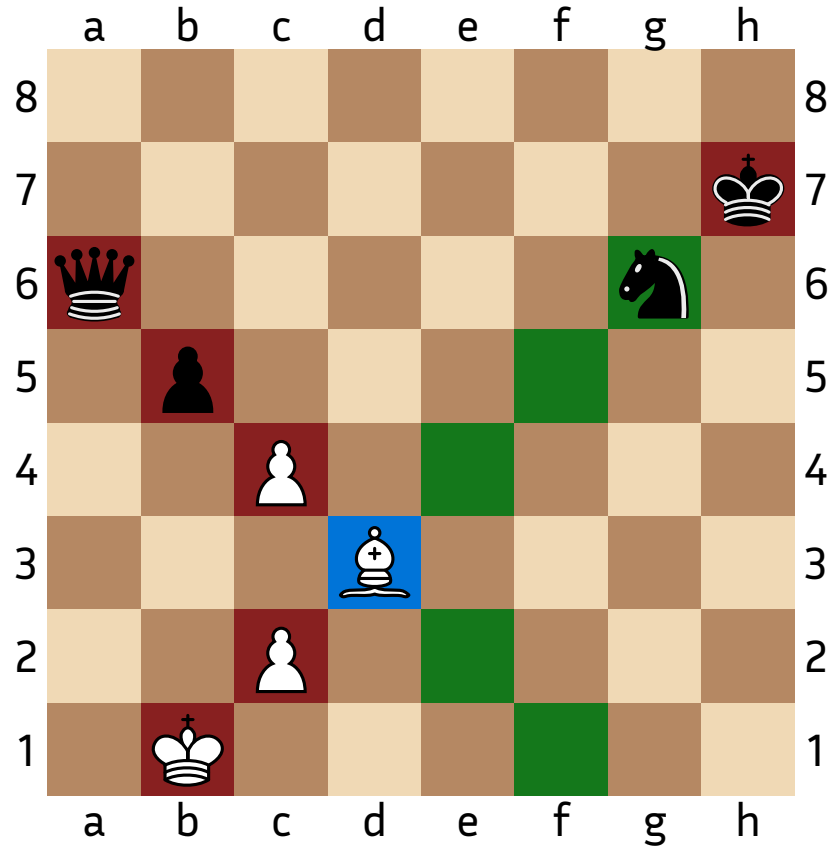
Generation des coups du fou



$$\forall (\varepsilon_1, \dots, \varepsilon_7) \in \{0, 1\}^7, M = \begin{pmatrix} \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \varepsilon_7 \\ \cdot & \varepsilon_5 & \cdot & \cdot & \cdot & \cdot & \varepsilon_6 & \cdot \\ \cdot & \cdot & \varepsilon_3 & \cdot & \varepsilon_4 & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & f & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \varepsilon_1 & \cdot & \varepsilon_2 & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \end{pmatrix}$$

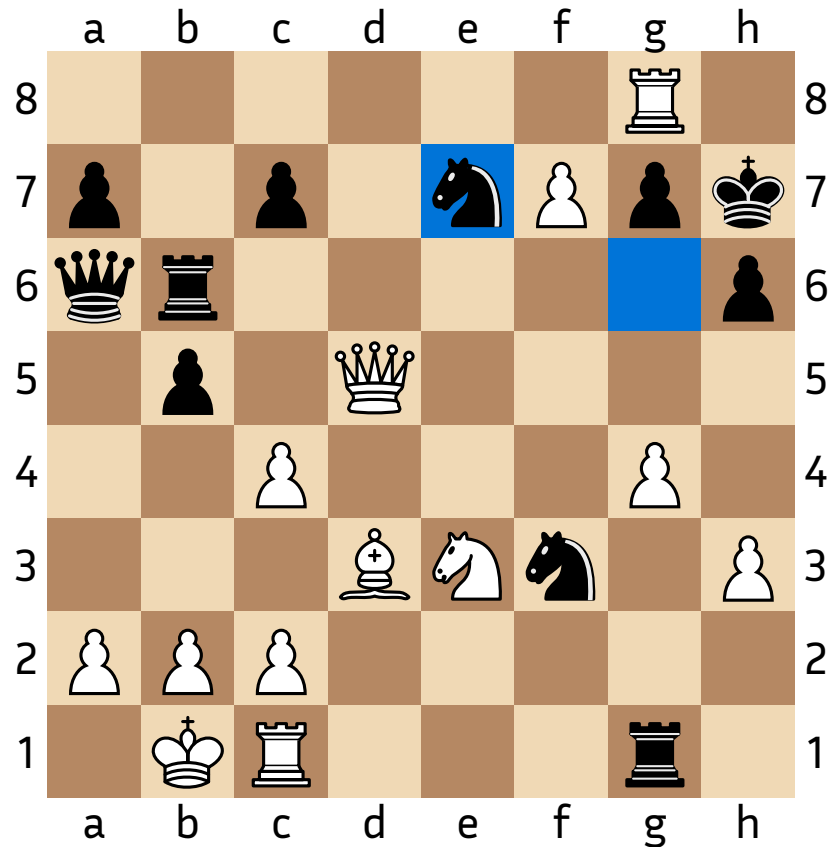
On génère les cases accessibles si M était la matrice des pièces bloquantes

Generation des coups du fou



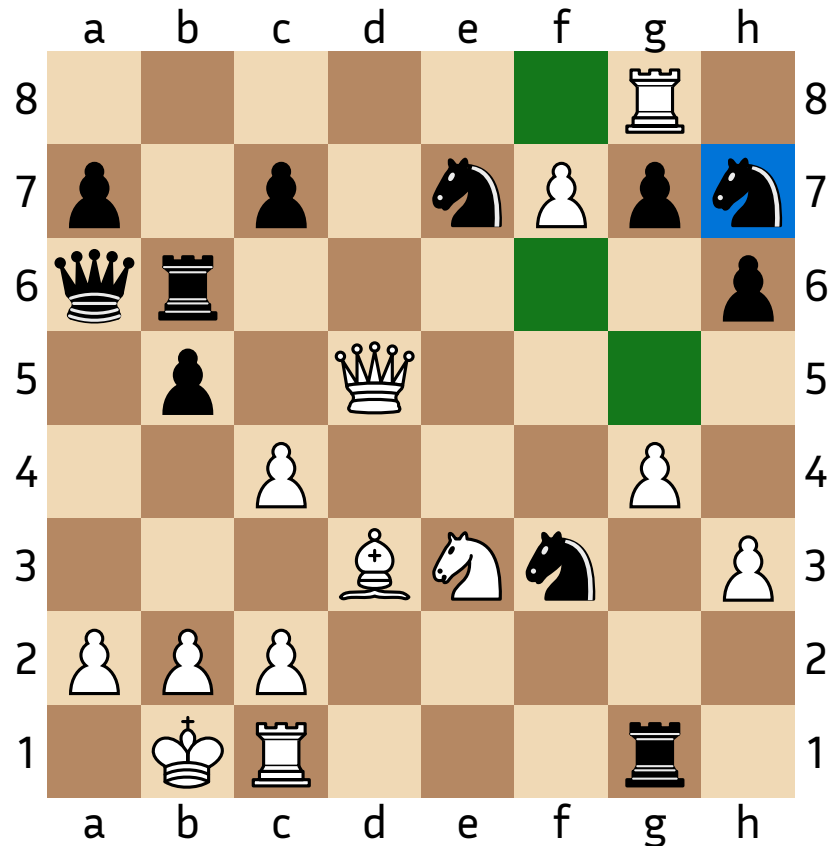
$$\begin{pmatrix}
 \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\
 \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\
 \cdot & 1 & \cdot & \cdot & \cdot & \cdot & 1 & \cdot \\
 \cdot & \cdot & 1 & \cdot & \cdot & \cdot & \cdot & \cdot \\
 \cdot & \cdot & \cdot & f & \cdot & \cdot & \cdot & \cdot \\
 \cdot & \cdot & 1 & \cdot & \cdot & \cdot & \cdot & \cdot \\
 \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\
 \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot
 \end{pmatrix}
 \rightarrow
 \underbrace{\begin{pmatrix}
 \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\
 \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\
 \cdot & \cdot & \cdot & \cdot & \cdot & 1 & \cdot & \cdot \\
 \cdot & \cdot & 1 & \cdot & 1 & \cdot & \cdot & \cdot \\
 \cdot & \cdot & \cdot & f & \cdot & \cdot & \cdot & \cdot \\
 \cdot & \cdot & 1 & \cdot & 1 & \cdot & \cdot & \cdot \\
 \cdot & \cdot & \cdot & \cdot & \cdot & 1 & \cdot & \cdot \\
 \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot
 \end{pmatrix}}_A
 \rightarrow
 \underbrace{\begin{pmatrix}
 \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\
 \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\
 \cdot & \cdot & \cdot & \cdot & \cdot & 1 & \cdot & \cdot \\
 \cdot & \cdot & \cdot & \cdot & 1 & \cdot & \cdot & \cdot \\
 \cdot & \cdot & \cdot & f & \cdot & \cdot & \cdot & \cdot \\
 \cdot & \cdot & \cdot & \cdot & 1 & \cdot & \cdot & \cdot \\
 \cdot & \cdot & \cdot & \cdot & \cdot & 1 & \cdot & \cdot \\
 \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot
 \end{pmatrix}}_{B=A \wedge \neg M_b}$$

Vérification de la légalité des coups



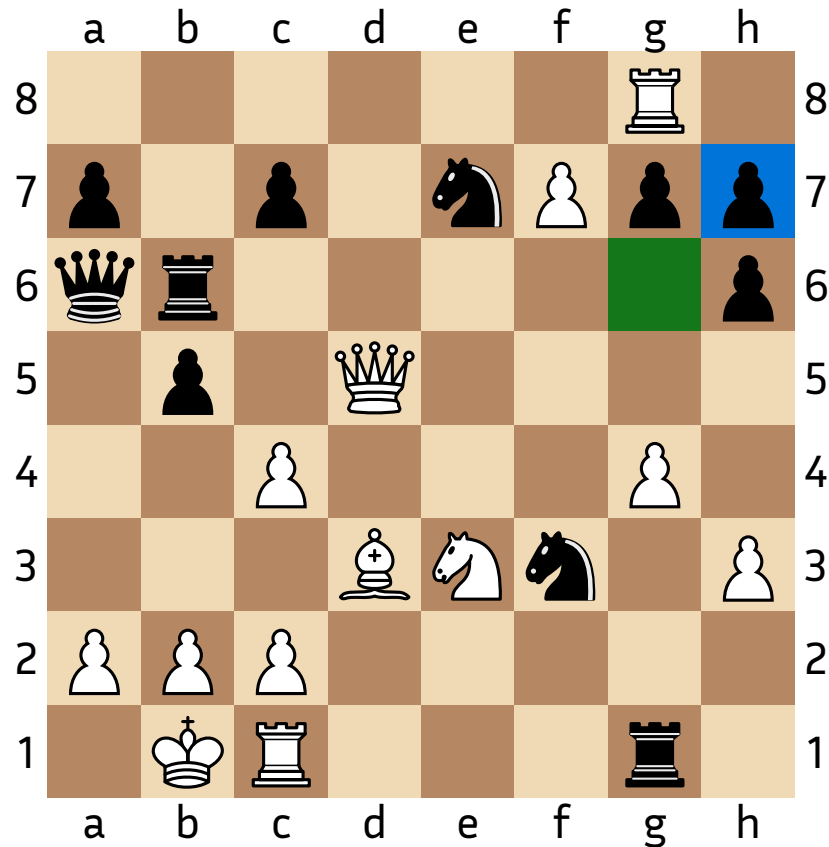
Pour vérifier la légalité des coups, l'algorithme remplace le roi par chaque type de pièces.

Vérification de la légalité des coups



Pour vérifier la légalité des coups, l'algorithme remplace le roi par chaque type de pièces.

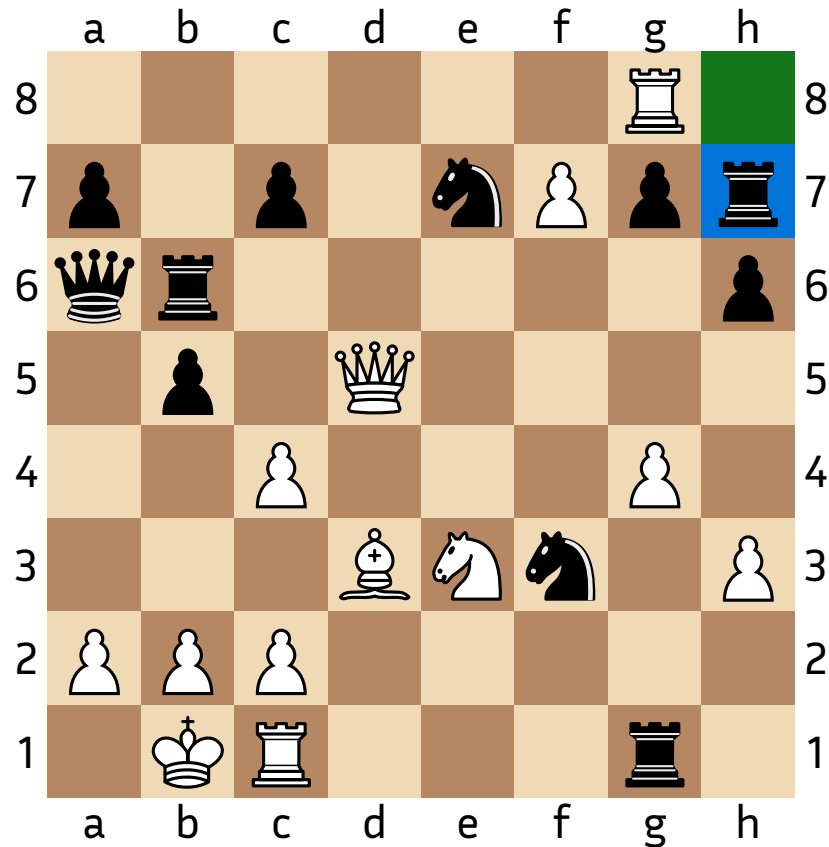
Pas de cavalier sur les cases colorées.



Pour vérifier la légalité des coups, l'algorithme remplace le roi par chaque type de pièces.

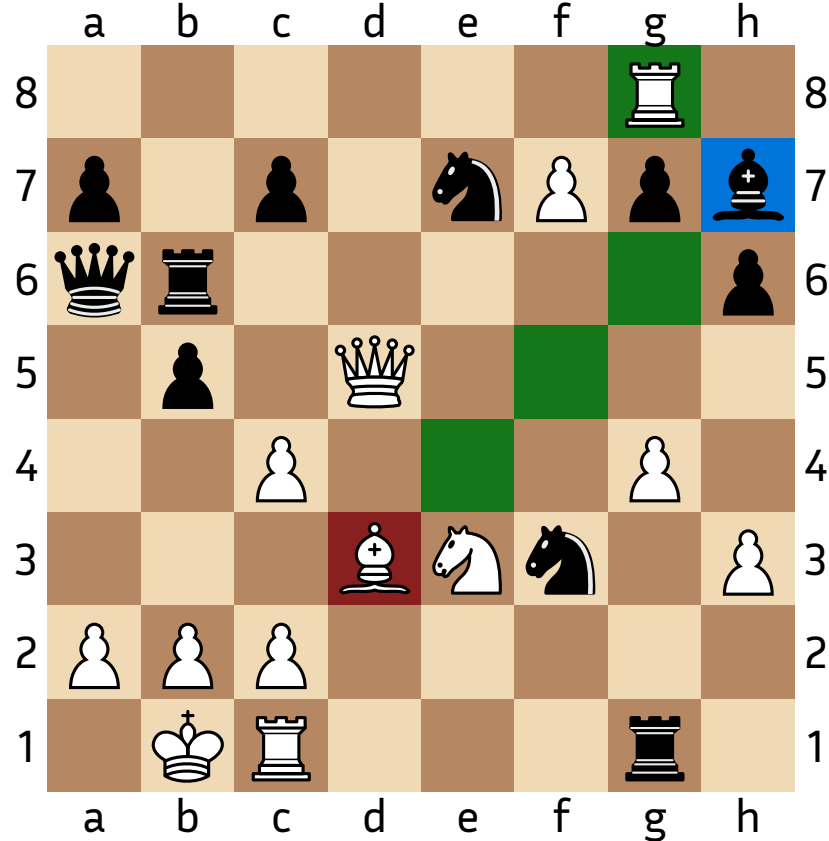
Pas de pion sur les cases colorées.

Vérification de la légalité des coups



Pour vérifier la légalité des coups, l'algorithme remplace le roi par chaque type de pièces.

Pas de tour ni de dame sur les cases colorées.



Pour vérifier la légalité des coups, l'algorithme remplace le roi par chaque type de pièces.

Pas de dame mais un fou sur les cases colorées,
le coup est illégal.

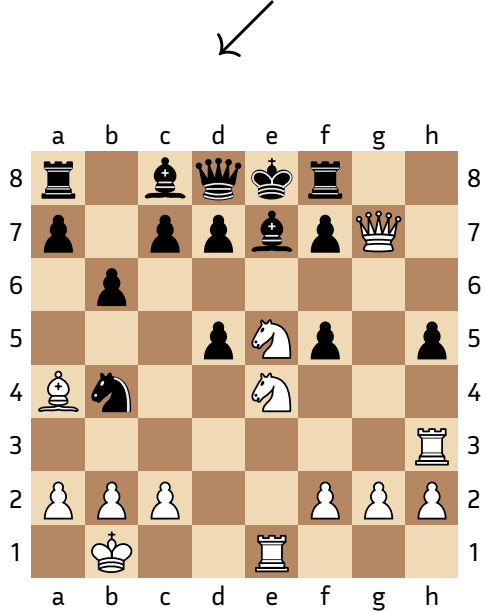
Recherche du mat

Ordres de grandeur

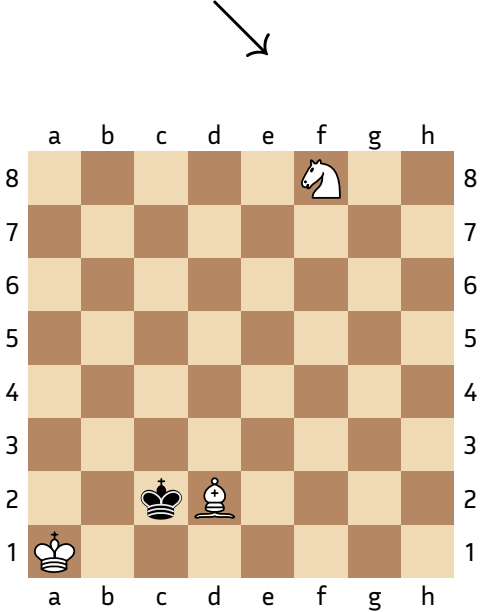
N	1	2	3	4
Temps	$\sim 100\mu s$	$\sim 15ms$	$\sim 3\text{ min}$	$+\infty$

Complexité exponentielle $O(b^{2N-1})$
b: facteur de branchement moyen,
ie nombre moyen de coups légaux par
position

Cas problématiques



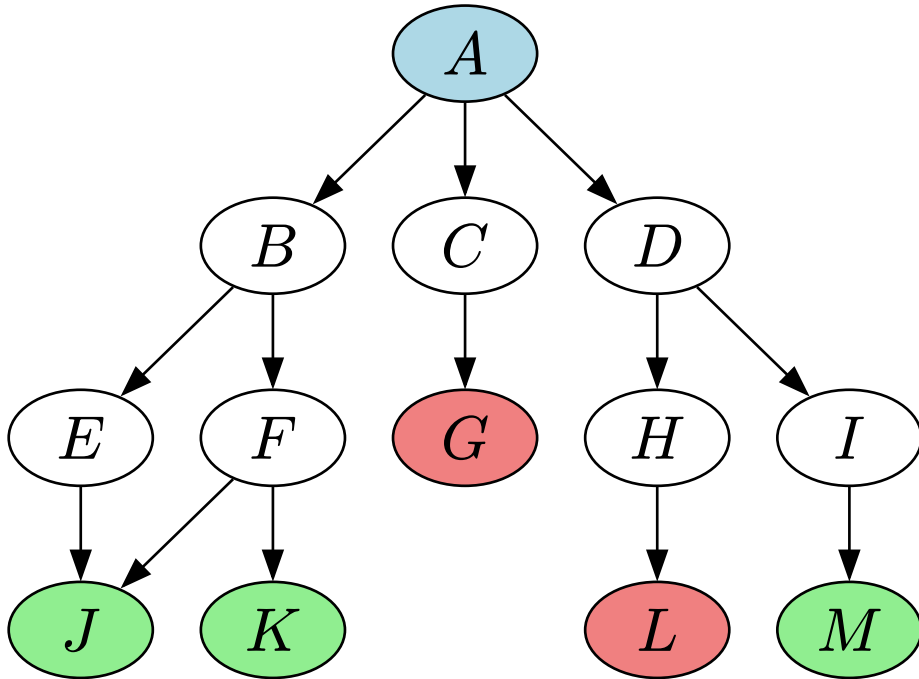
Mat en 3 coups



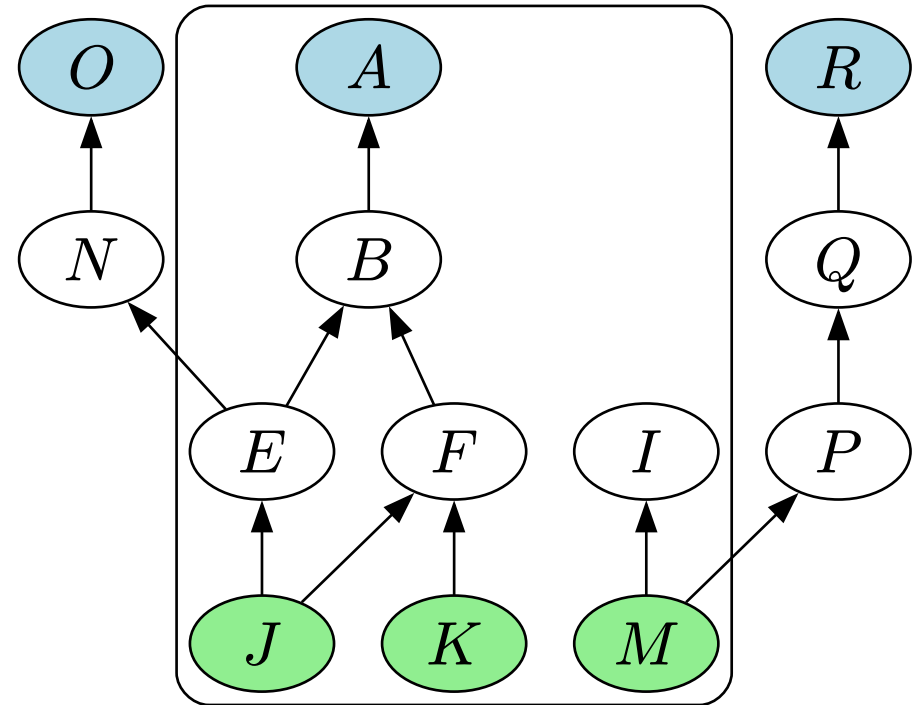
Mat en 33 coups

Fins de parties

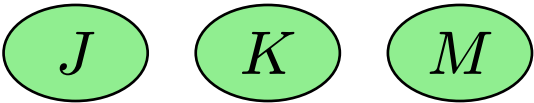
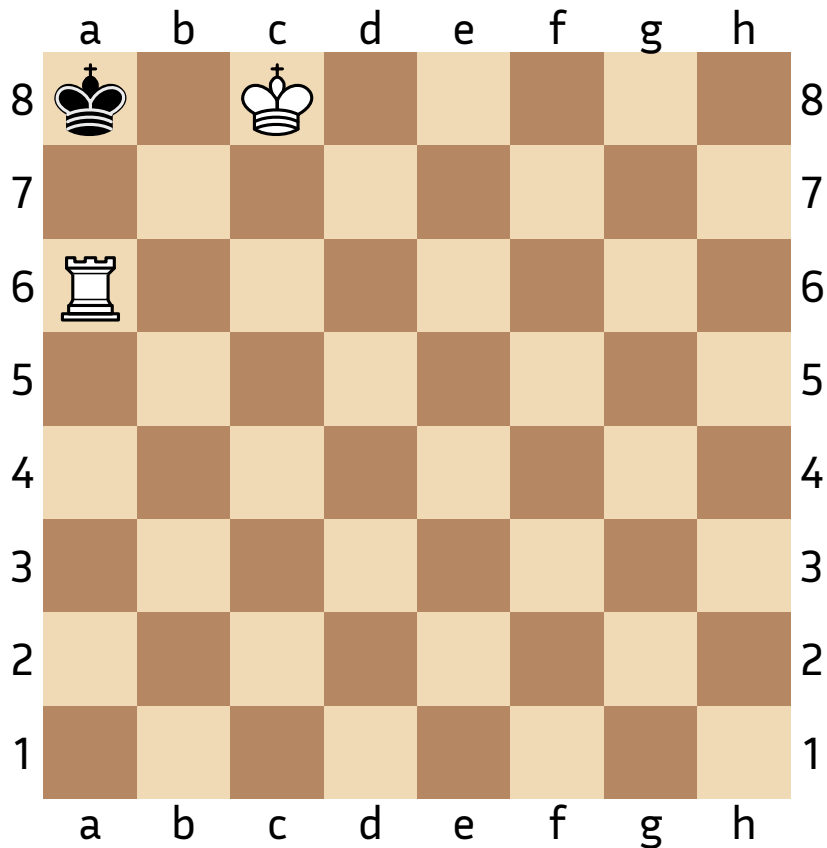
Méthode usuelle



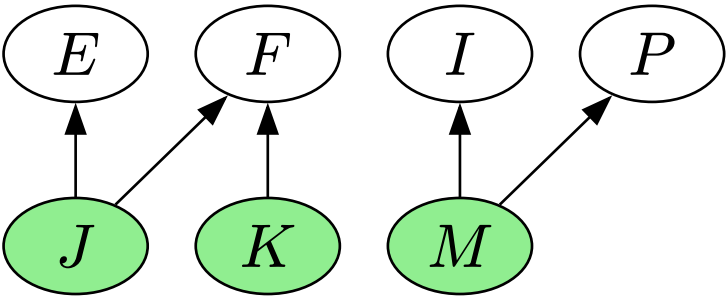
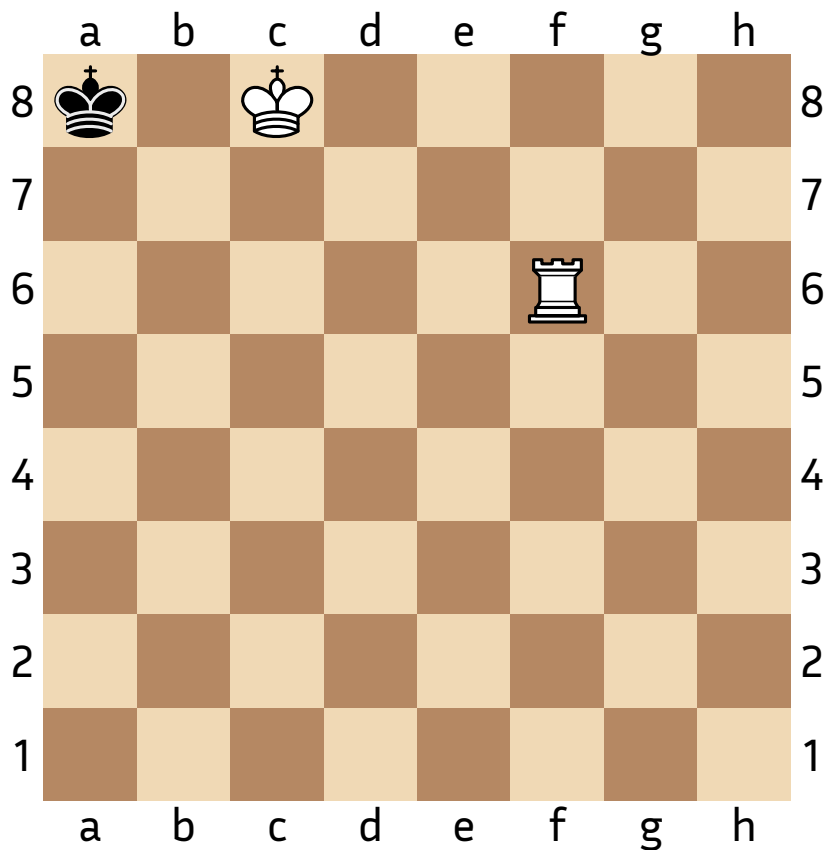
Méthode d'analyse rétrograde



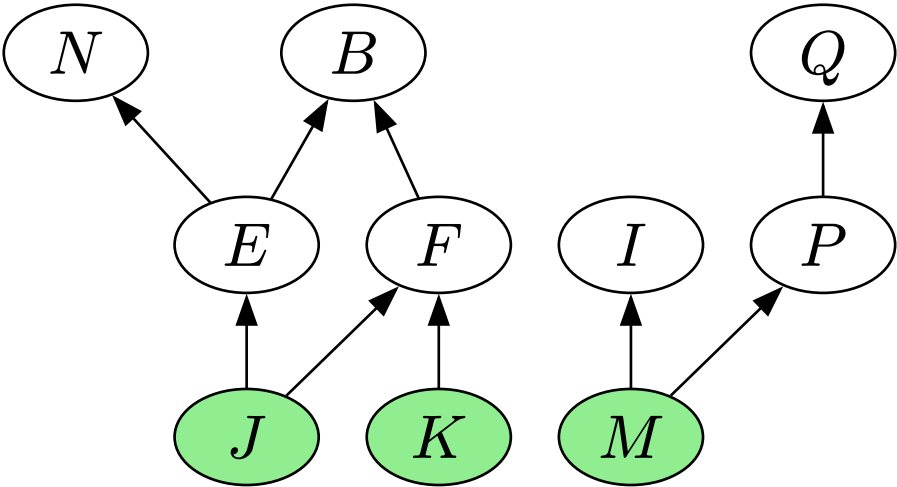
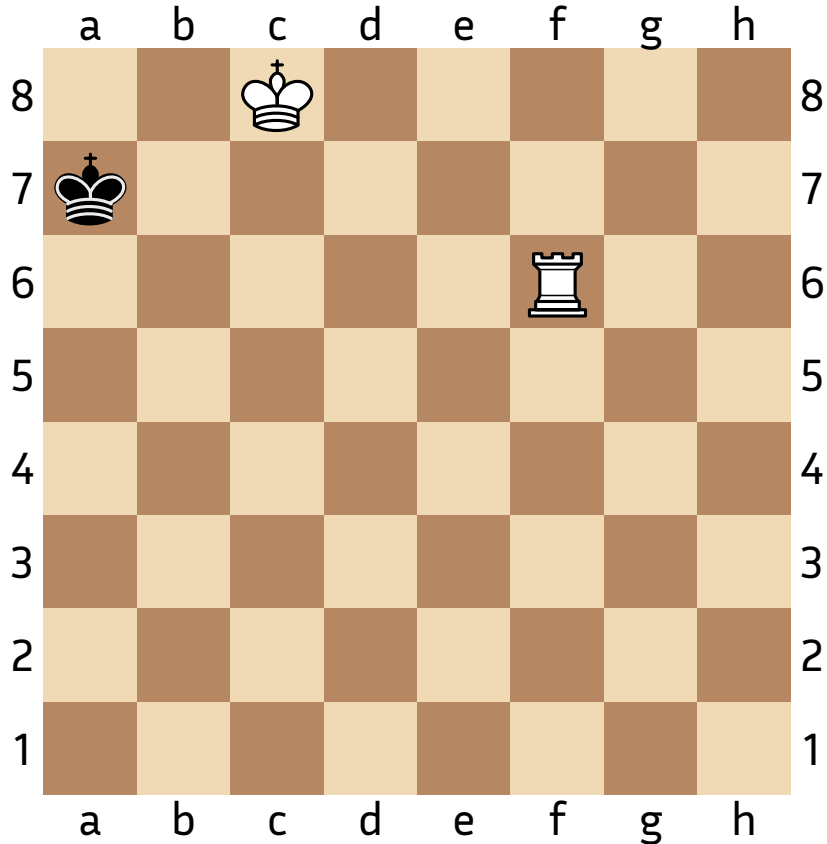
Analyse rétrograde



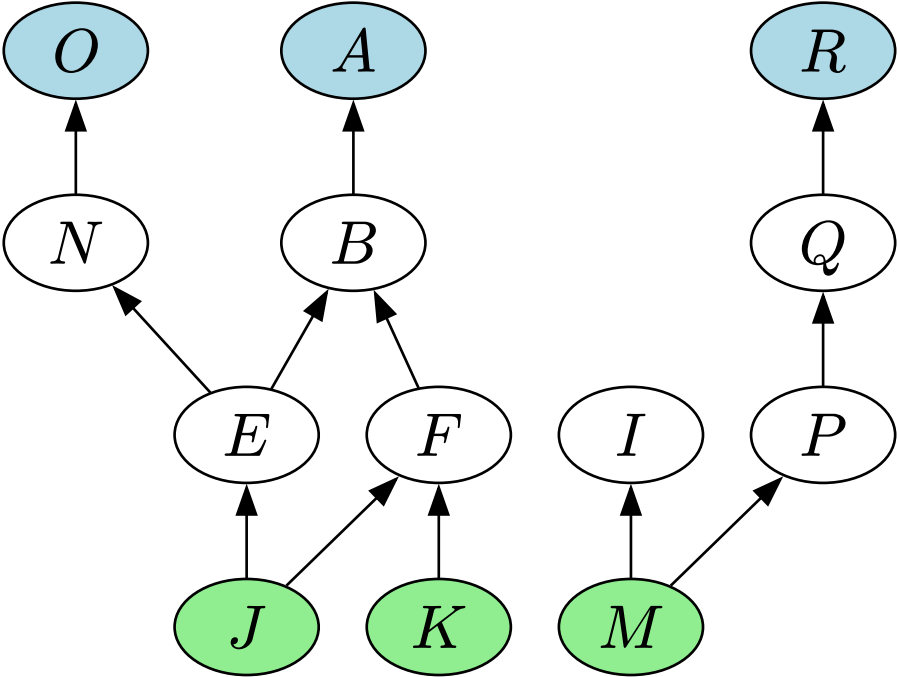
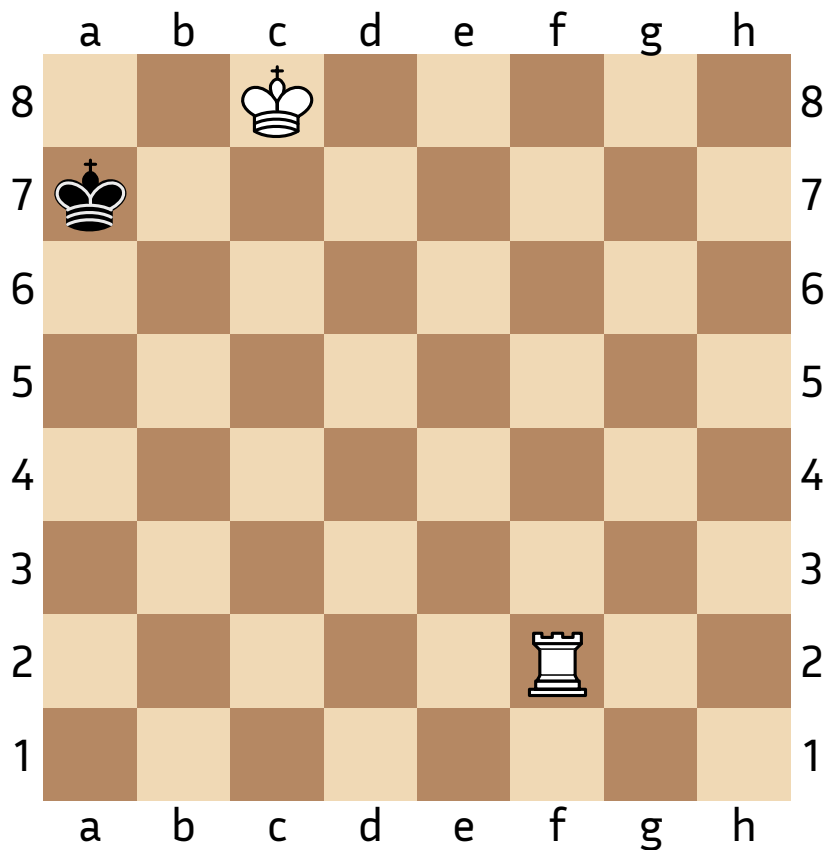
Analyse rétrograde



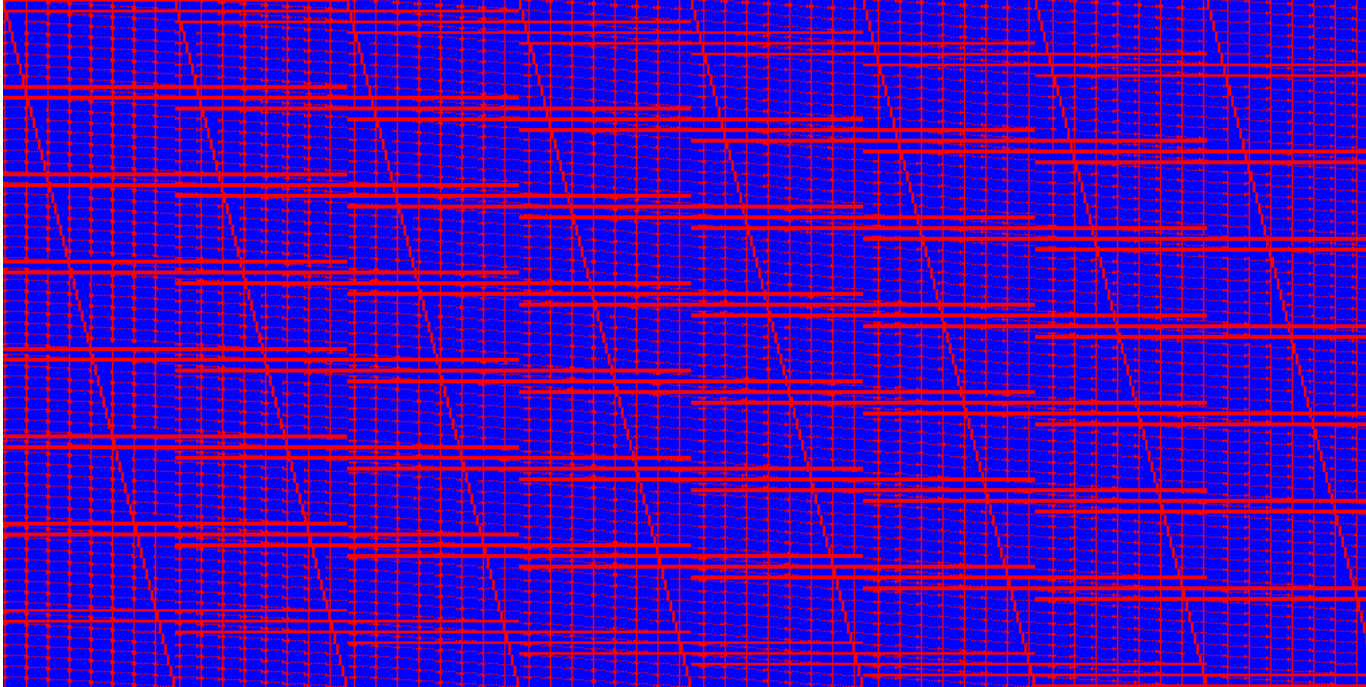
Analyse rétrograde



Analyse rétrograde



Optimisation



Taille du tableau:

$$64 \times 64 \times 64 \times 2 = 524\,288$$

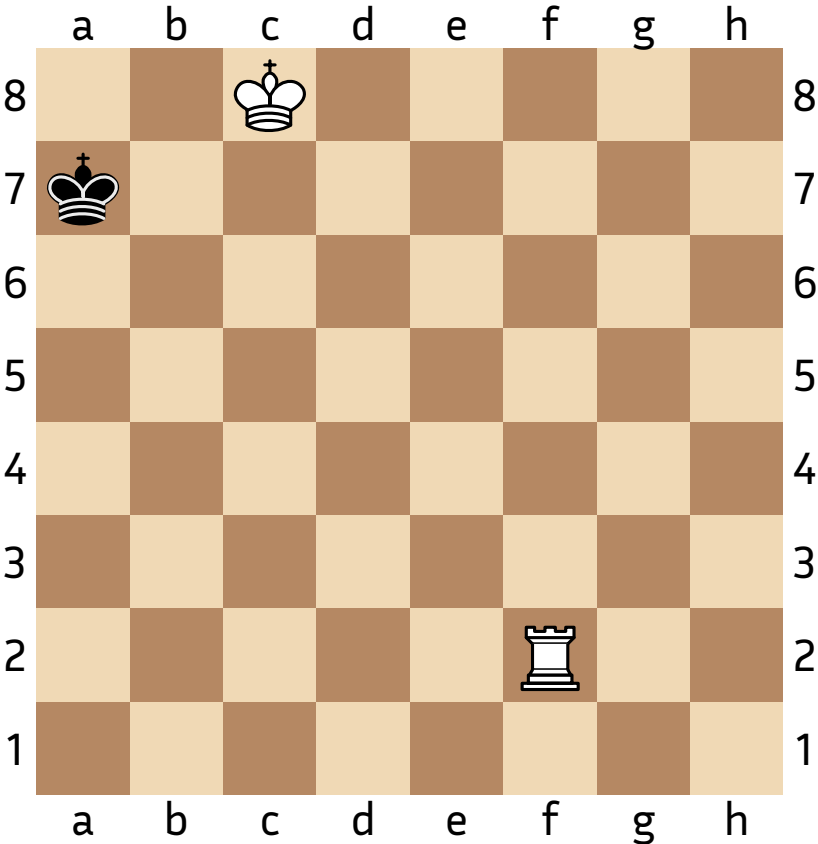
Espace de stockage: 524 Ko

Temps d'exécution: 8 s

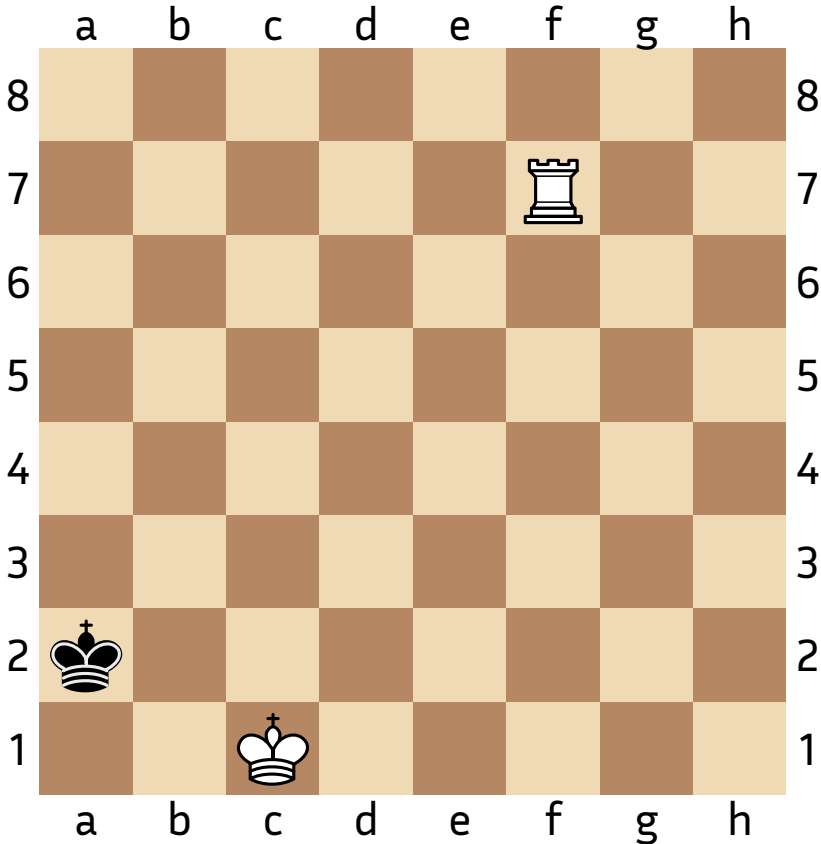
Densité: $d \sim 0.73$

Fig. 1. – Carte de chaleur: Roi, Tour vs Roi

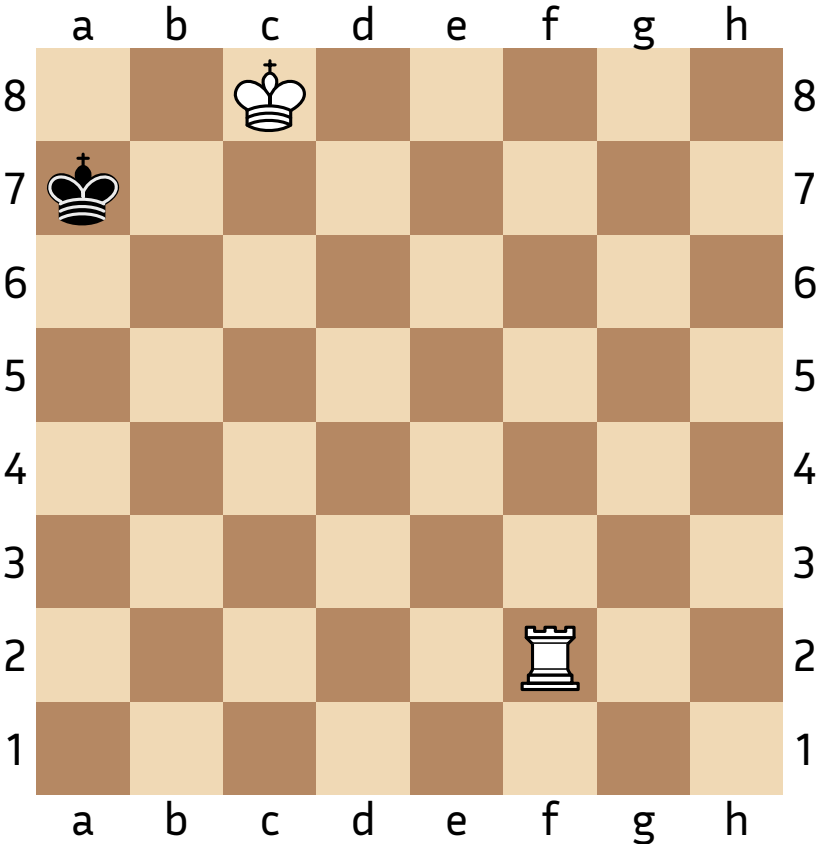
Optimisation



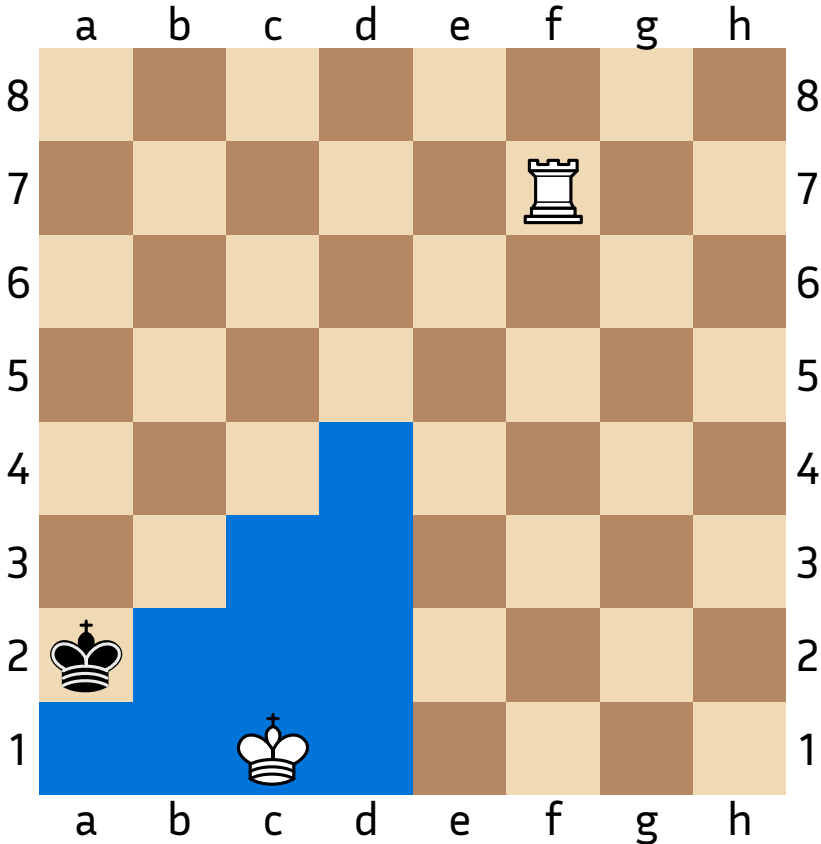
Sym
→



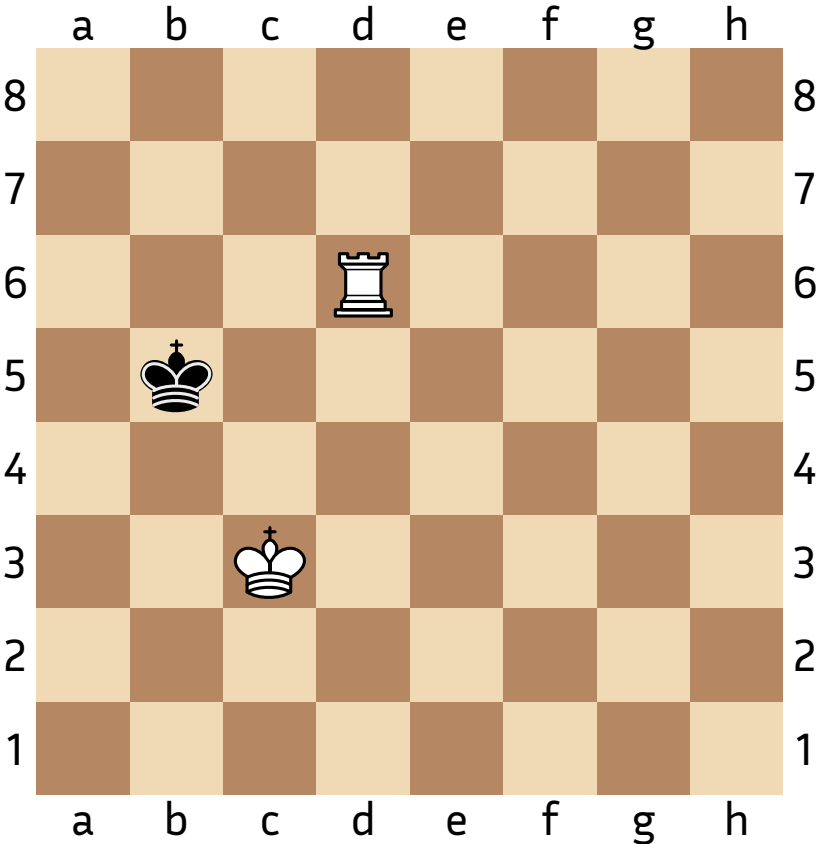
Optimisation



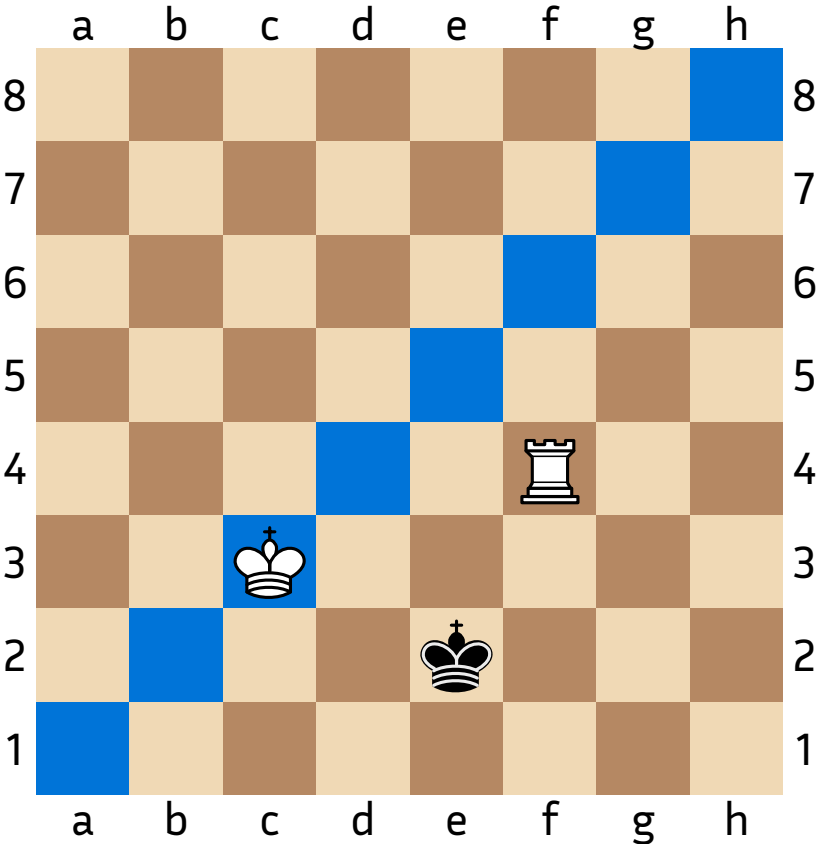
Sym
→



Optimisation



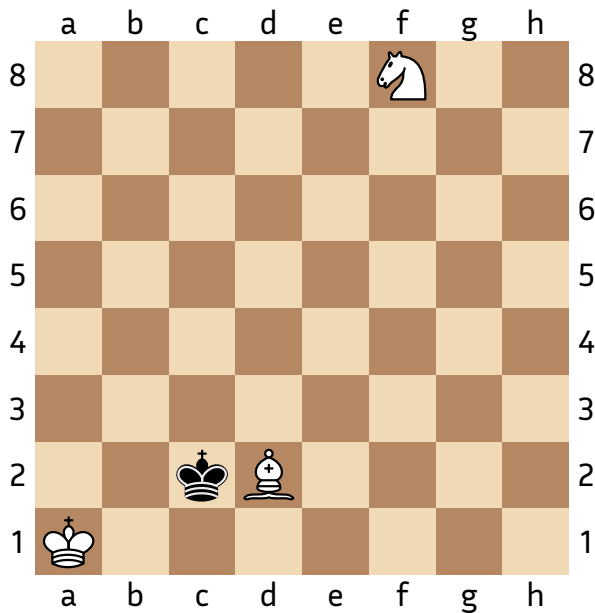
Sym
→



Représentation en machine

$$\mathcal{A} = \left\{ \text{Positions possibles utilisant le jeu de pieces } (p_i)_{i \in I} \right\}$$

On peut créer $f : \mathcal{A} \rightarrow \llbracket 0; 462 \times 64^{|I|} \times 2 - 1 \rrbracket$ injective



$$((6 \times 64 + 11) \times 64 + 61) \times 2 + 1 = 50683$$

```

indice = position rois
Pour chaque piece
    indice = indice * 64 + position
Fin Pour
indice = indice * 2 + joueur
    
```

$$f(\text{position}) = ((i_{\text{rois}} \times 64 + i_{\text{fou}}) \times 64 + i_{\text{cavalier}}) \times 2 + i_{\text{joueur}}$$

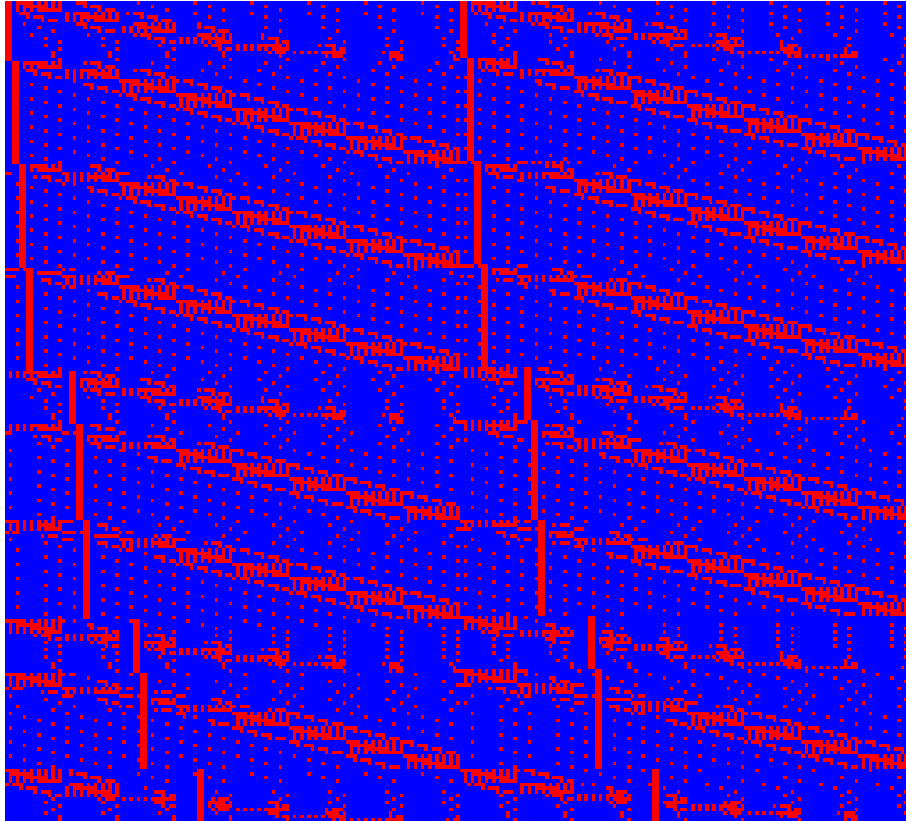
$i_{\text{rois}} \in \llbracket 0; 461 \rrbracket$, indice des deux rois

$i_{\text{fou}} \in \llbracket 0; 63 \rrbracket$, indice du fou

$i_{\text{fou}} \in \llbracket 0; 63 \rrbracket$, indice du cavalier

$i_{\text{joueur}} \in \llbracket 0; 1 \rrbracket$, joueur actif (0 pour les Blancs, 1 pour les Noirs)

Exploitation des résultats



Taille du tableau: $462 \times 64 \times 2 = 59\,136$

Espace de stockage: 59 Ko

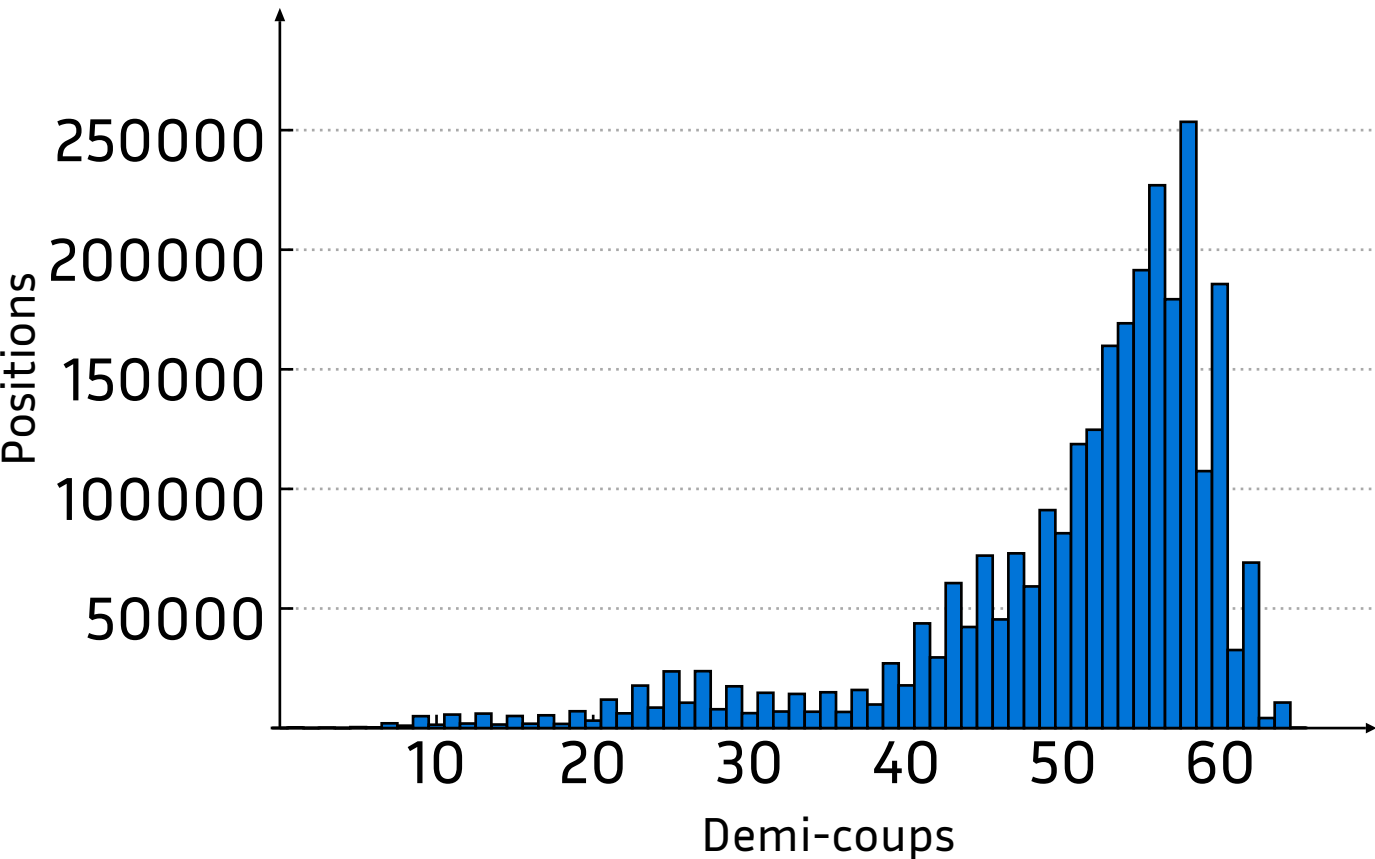
Temps d'exécution: 1180 ms

Densité: $d \sim 0.82$

Réduction de la taille par 9

Fig. 2. – Carte de chaleur: Roi, Tour vs Roi

Exploitation des résultats



- Espace de stockage: 3.8 Mo
- Temps d'exécution: 1 min 18s

Graph 1. – Roi, Fou, Cavalier vs Roi

Vérification des résultats

- Utilisation des travaux de Ronald de Man
 - tables SYZYGY
- Tests d'intégrité de la table de finales



Conclusion

- Algorithme efficace car les pré-calculs permettent une exécution très rapide
- Limites
 - Phase d'initialisation qui nécessite beaucoup de ressources CPU et stockage
 - Nombre de pièces sur l'échiquier / Stockage et temps d'exécution

Références bibliographiques

- [1] CLAUDE SHANNON : Programming a Computer for Playing Chess : Philosophical Magazine, Ser.7, Vol. 41, No. 314 - March 1950
- [2] VUČKOVIĆ VLADAN V : Realization of the chess mate solver application : Faculté de génie électronique, Université de Niš, Niš, Serbie et Monténégro, Reçu : juillet 2003 / Accepté : Juin 2004, 16 pages
- [3] PIETER BIJL, ANH PHI TIET : Exploring modern chess engine architectures : Vrije Universiteit Amsterdam, 30 pages, 5 juillet 2021
- [4] MARK LEFLER : Chessprogramming : <https://www.chessprogramming.org>
- [5] KEN THOMPSON : Retrograde Analysis Of Certain Endgames : ICCA Journal, Vol. 9, No. 3 (September 1986)
- [6] LÁSZLÓ POLGÁR : 5334 Problems, Combinations and Games : Black Dog & Leventhal, 978-1579125547
- [7] RONALD DE MAN : Syzygy endgame tablebases : <https://syzygy-tables.info>